

Market Lens: ML-Driven Stock Analysis System

Half-Term Progress Report

Amatya Simba Akshat Prakash Singh Aayush A Sapra
Software Engineering Lab, CSED

September 17, 2026

1. Project Recap

Market Lens is a web platform that consolidates stock research, trend analysis, and ML-based recommendations into one tool for retail investors and finance students, focused initially on the Indian stock market. The core objective is to move beyond fragmented brokerage apps, news sites, and spreadsheets by combining live/historical price data, trend visualization, personalized ML-driven outlook scoring, and a risk-free paper-trading simulator. Development follows weekly Agile Scrum sprints, prioritizing a stable core (auth, search, charting) before layering ML capability on top.

2. Progress to Date

Work so far has focused on establishing the core pipeline and delivering the first version of the ML-driven recommendation feature ahead of the original schedule:

- **Core scaffolding:** Project setup, authentication flow, and dashboard UI are in place, consistent with the Sprint 1 plan (scaffolding, wireframes, schema, JWT auth, proof-of-concept API call).
- **ML prediction module (key milestone):** The dashboard now supports **CSV upload-based prediction** — a user can upload a CSV of historical price data and the system trains/fits a model on it on the fly to forecast future stock pricing, rather than relying solely on a fixed pre-trained model.
- **Market scope:** The current implementation and testing have been carried out exclusively on **Indian stocks**, matching the project's original market focus.

This effectively pulls forward part of the "enhanced" ML pipeline deliverable that was originally scoped for after the core (charting/watchlist) was stable, since the prediction feature was validated early using user-supplied data rather than waiting on live-API integration.

3. Current Architecture

The system remains a standard 3-tier, stateless design so it stays maintainable while ML capability grows:

- **Frontend:** React.js with a charting library (Chart.js/Recharts) and Tailwind CSS, including the new CSV-upload interface for the prediction module.
- **Backend:** Node.js/Express (or Django) REST API handling auth, caching, feature engineering, and the model training/prediction pipeline triggered by CSV upload.
- **Data:** PostgreSQL/MongoDB for users, portfolios, and transactions; market data sourced from Alpha Vantage/Yahoo Finance (cached) for non-uploaded flows.

4. Evaluation Against Original Criteria

- *Response time* (target: median $\leq 2s$ for a recommendation): being measured as the CSV-based prediction path is load-tested; not yet finalized.
- *Prediction usability:* CSV-driven training gives a working, testable outlook signal for Indian stocks, satisfying the near-term goal of validating the ML approach before broader rollout.
- *Scope discipline:* the CSV-upload approach was chosen deliberately to de-risk ML complexity — in line with the plan to start from a simple baseline before more advanced models.

5. Deviations and Risks

- **Scope shift:** ML prediction (CSV-based) was accelerated relative to plan; historical charting and watchlist features are correspondingly slightly behind their original sprint target.
- **Market coverage:** Indian-stock-only scope is intentional for this phase but limits current generalizability; broader markets remain a stretch goal.
- **Data quality risk:** arbitrary user-uploaded CSVs introduce a new risk — inconsistent formats or insufficient history could degrade prediction quality. Input validation and format guidance are planned.
- **API rate limits and scope creep** remain tracked per the original mitigation plan (caching, mock-data fallback, scope freeze after Sprint 3).

6. Next Steps

1. Harden CSV input handling (validation, minimum data requirements, error messaging).
2. Complete historical charting (line/candlestick) and watchlist features to close the core-feature gap.
3. Integrate live/cached market data alongside CSV upload so predictions are not upload-dependent only.
4. Begin paper-trading simulation build-out, feeding from the same prediction pipeline.
5. Formal load-testing of the response-time evaluation criterion.